Basic REST API Interview Questions

These questions test your foundational knowledge. Expect them in almost every interview, regardless of seniority level.

1. What is REST and what are its constraints?

REST stands for Representational State Transfer. It is an architectural style for designing networked applications, defined by Roy Fielding in his 2000 doctoral dissertation.

REST has six constraints:

1. **Client-Server:** Separation of concerns between the user interface and data storage
2. **Stateless**: Each request contains all information needed to complete it
3. **Cacheable:** Responses must indicate whether they can be cached
4. **Layered System**: Components cannot see beyond their immediate layer
5. **Uniform Interface:** Standardized way to interact with resources
6. **Code-on-Demand (optional):** Servers can send executable code to clients

2. What is the difference between REST and SOAP?

REST is an architectural style; SOAP is a protocol. REST typically uses JSON over HTTP and leverages standard HTTP methods. SOAP uses XML exclusively and defines its own messaging format with envelopes and headers.

Aspect	REST	SOAP
Format	JSON, XML, others	XML only
Transport	HTTP	HTTP, SMTP, others
Caching	Native HTTP caching	Not cacheable
Performance	Lightweight	Higher overhead

SOAP still makes sense for enterprise systems requiring formal contracts (WSDL) or message-level security (WS-Security).

3. What are HTTP methods and when do you use each?

HTTP methods define the action to perform on a resource.

Method	Purpose	Example
GET	**Retrieve a resource**	**GET /users/123**
POST	**Create a new resource**	**POST /users**
PUT	**Replace a resource entirely**	**PUT /users/123**
PATCH	**Partially update a resource**	**PATCH /users/123**
DELETE	**Remove a resource**	**DELETE /users/123**

4. What is the difference between PUT and POST?

POST creates a new resource where the server determines the URI. PUT replaces a resource at a specific URI that the client specifies.

```
POST /users          → Server creates user, returns /users/123
```

```
PUT /users/123       → Client specifies URI, replaces entire resource
```


If you PUT to a URI that does not exist, the server may create it. If you PUT to an existing URI, you replace the entire resource.

5. What is the difference between PUT and PATCH?

PUT replaces the entire resource. PATCH applies partial modifications.

```
# PUT replaces everything
```

```
PUT /users/123
```

```
{"name": "Khalid", "email": "khalid@example.com", "role": "admin"}
```

```
# PATCH updates only specified fields

PATCH /users/123

{"role": "admin"}
```

Powered By

6. What does "stateless" mean in REST?

Stateless means each request must contain all the information the server needs to process it. The server does not store any client context between requests.

This means no server-side sessions. If a user is authenticated, the client must send credentials (typically a token) with every request.

7. What makes an API RESTful?

An API is truly RESTful when it satisfies all REST constraints, including HATEOAS (Hypermedia as the Engine of Application State). In practice, most APIs that call themselves "REST" are actually "REST-like" or at Richardson Maturity Model Level 2. And honestly, that is fine for most use cases.

Richardson Maturity Model levels for REST APIs. Image by Author.

8. What are the basic HTTP status codes you should know?

You should be familiar with at least these seven essential status codes that cover the majority of API scenarios:

Code	Name	When to Use
200	OK	Successful GET, PUT, PATCH
201	Created	Successful POST (include Location header)
204	No Content	Successful DELETE
400	Bad Request	Malformed syntax
404	Not Found	Resource does not exist
500	Internal Server Error	Server-side failure

9. What is the difference between a resource and an endpoint?

A resource is the data entity (user, order, product). An endpoint is the URL path that provides access to that resource.

`Resource: User`

`Endpoints: GET /users, GET /users/123, POST /users`

10. What are best practices for URI design?

Use nouns, not verbs. Keep URIs lowercase with hyphens for multi-word names. Use plural nouns for collections.

`Good:  /users, /users/123, /order-items`

`Bad:  /getUsers, /Users, /order_items`

Avoid deep nesting. Instead of /users/123/posts/456/comments/789, consider /comments/789 or /comments?post_id=456.

11. What is idempotency and which HTTP methods are idempotent?

An operation is idempotent if performing it multiple times produces the same result as performing it once.

Method	Idempotent	Why
GET	**Yes**	**Reading does not change state**
PUT	**Yes**	**Replacing with same data yields same result**
DELETE	**Yes**	**Deleting twice leaves resource deleted**
POST	**No**	**Creating twice creates two resources**
PATCH	**No***	**Depends on the operation**

A PATCH that sets a value is idempotent; a PATCH that increments a counter is not.

12. What is the difference between safe and idempotent methods?

Safe methods do not modify resources (GET, HEAD, OPTIONS). Idempotent methods can be called multiple times with the same effect (GET, PUT, DELETE).

All safe methods are idempotent, but not all idempotent methods are safe. DELETE is idempotent but not safe because it modifies state.

13. When should you use query parameters versus path parameters?

Path parameters identify a specific resource. Query parameters filter, sort, or paginate collections.

```
Path:   /users/123          (specific user)

Query:  /users?status=active    (filtered collection)

        /users?sort=name&limit=10
```


14. What is content negotiation?

Content negotiation allows clients to request different representations of a resource. The client sends an Accept header, and the server responds with the appropriate format.

```
Accept: application/json    → Server returns JSON

Accept: application/xml     → Server returns XML
```


15. What is the purpose of the OPTIONS method?

OPTIONS returns the HTTP methods that a server supports for a specific URL. Browsers use it for CORS preflight requests to check if cross-origin requests are allowed.

Intermediate REST API Interview Questions

These questions test practical application and decision-making. Expect them for mid-level positions.

16. What is the difference between 401 and 403?

This is the most commonly confused pair of status codes. I have seen senior engineers mix these up in code reviews.

401 Unauthorized

Means unauthenticated. The server does not know who you are. Your response should prompt the user to log in.

403 Forbidden

Means unauthorized. The server knows who you are but you lack permission. Re-authenticating will not help.

Authentication versus authorization error decision flowchart diagram. Image by Author.

17. What is the difference between 400 and 422?

Here are examples illustrating the difference:

400 Bad Request

This indicates malformed syntax. The server cannot parse the request (invalid JSON structure, missing required headers).

422 Unprocessable Entity

This indicates valid syntax but semantic errors. The JSON is valid but the data fails validation (invalid email format, password too short).

```
# 400: Cannot parse

{"name": "Khalid",}  # Trailing comma breaks JSON


# 422: Valid JSON, invalid data

{"email": "not-an-email"}  # Fails validation
```

Powered By

18. When should you use 409 Conflict?

Use 409 when the request conflicts with the current state of the resource:

- Optimistic locking failures (ETag mismatch)
- Unique constraint violations (duplicate username)
- Invalid state transitions (canceling an already-shipped order)

19. How do you implement pagination?

Two main approaches:

Offset-based pagination

Simple but has performance issues with large datasets.

```
GET /users?limit=20&offset=40
```

Powered By

Cursor-based pagination

Scales better and avoids "page drift" when data changes.

```
GET /users?limit=20&cursor=eyJpZCI6MTIzfQ
```

Powered By

Companies like Stripe, GitHub, and Slack use cursor-based pagination for large datasets.

20. What are common API versioning strategies?

There are three main approaches, each with different trade-offs:

URI versioning

This is the most common. It is explicit and easy to debug.

```
GET /v1/users
```

```
GET /v2/users
```

Powered By

Header versioning

Cleaner URLs but harder to test in browsers.

```
GET /users
```

```
Accept-Version: v2
```

Powered By

Query parameter versioning

Easy to add but clutters URLs.

```
GET /users?version=2
```

Powered By

21. What is an ETag and how does caching work?

An ETag is a version identifier for a resource. The server sends it in responses; the client sends it back in subsequent requests to check if the resource changed.

```
# First request
```

```
GET /users/123
```

```
Response: ETag: "abc123"
```

```
# Subsequent request
GET /users/123
If-None-Match: "abc123"
Response: 304 Not Modified (if unchanged)
```


22. What authentication methods are used for REST APIs?

The four most common authentication methods are API Keys, Bearer Tokens, OAuth 2.0, and JWT:

Method	Use Case	Pros	Cons
API Keys	**Server-to-server**	**Simple**	**No expiration, easy to leak**
Bearer Tokens	**Session auth**	**Stateless**	**Must secure storage**
OAuth 2.0	**Third-party access**	**Delegated auth**	**Complex implementation**
JWT	**Stateless auth**	**Self-contained**	**Cannot revoke without blacklist**

23. What is JWT and how does it work?

JWT (JSON Web Token) consists of three parts: Header, Payload, and Signature. Learn more about **working with JSON in Python**.

```
Header: {"alg": "RS256", "typ": "JWT"}
```

```
Payload: {"sub": "user123", "exp": 1704153600}

Signature: RSASHA256(base64(header) + "." + base64(payload), privateKey)
```

Powered By

The server signs the token; clients send it with requests. The server validates the signature without database lookups.

24. What is the difference between RS256 and HS256?

HS256 (symmetric): Same secret signs and verifies. Risk if multiple services need to verify tokens.

RS256 (asymmetric): Private key signs, public key verifies. Recommended for distributed systems where multiple services validate tokens.

25. What is HATEOAS?

HATEOAS (Hypermedia as the Engine of Application State) means responses include links to related actions and resources.

```
{
 "id": 123,
 "name": "Khalid",
 "links": [
  {"rel": "self", "href": "/users/123"},
  {"rel": "orders", "href": "/users/123/orders"}
 ]
}
```

Powered By

In practice, most APIs do not implement HATEOAS fully. Know the concept but do not expect to implement it in most jobs. Even large tech companies rarely build fully HATEOAS-compliant APIs.

26. How do you handle errors consistently?

Use RFC 7807 Problem Details format for consistent error responses:

```
{
  "type": "https://api.example.com/errors/validation",
  "title": "Validation Error",
  "status": 422,
  "detail": "Email format is invalid",
  "instance": "/users"
}
```

Powered By

27. What is rate limiting and why is it important?

Rate limiting protects your API from abuse and ensures fair usage. Common response:

```
HTTP/1.1 429 Too Many Requests
Retry-After: 3600
X-RateLimit-Limit: 1000
X-RateLimit-Remaining: 0
```

Powered By

28. How do you design filtering and sorting?

Use query parameters with clear conventions:

```
# Filtering
GET /products?category=electronics&price[gte]=100&price[lte]=500


# Sorting (prefix - for descending)
GET /users?sort=-created_at,name
```

Powered By

Advanced REST API Interview Questions

These questions test architectural thinking. Expect them for senior positions.

29. What changed between OAuth 2.0 and OAuth 2.1?

OAuth 2.1 consolidates security best practices:

Feature	OAuth 2.0	OAuth 2.1
PKCE	**Optional**	**Mandatory for ALL clients**
Implicit Grant	**Supported**	**Removed**
Password Grant	**Supported**	**Removed**
Redirect URI	**Flexible matching**	**Exact match required**

The key takeaway: use Authorization Code flow with PKCE for all client types.

30. How do you implement idempotency for payment endpoints?

Use idempotency keys to make POST requests safe for retries:

```
async def handle_payment(request: Request, payment: PaymentCreate):
    idempotency_key = request.headers.get("Idempotency-Key")

    # Check if already processed
    cached = await redis.get(f"idem:{idempotency_key}")
    if cached:
        return JSONResponse(json.loads(cached))

    # Process payment
    result = await process_payment(payment)
```

```
    # Cache result (24-hour TTL)
    await redis.setex(f"idem:{idempotency_key}", 86400, json.dumps(result))
    return result
```

Powered By

The client generates a UUID and sends it with the request. If the same key arrives again, return the cached response.

31. How would you design a distributed rate limiter?

For distributed systems, you cannot use in-memory counters because traffic is load-balanced across instances. This is a common gotcha that trips up candidates.

Architecture:

1. Use Redis for centralized counter storage
2. Implement Token Bucket algorithm for burst tolerance
3. Use atomic operations (Lua scripts) to prevent race conditions

```
# Pseudocode for Token Bucket
tokens = redis.get(user_id) or max_tokens
if tokens > 0:
    redis.decr(user_id)
    process_request()
else:
    return 429
```

Powered By

32. When would you choose gRPC over REST?

The choice between gRPC and REST depends on your specific requirements.

Choose gRPC for:

- Internal microservice communication (5-10x throughput improvement)
- Real-time streaming requirements
- Mobile-to-backend with bandwidth constraints

- Polyglot environments needing strict contracts

Choose REST for:

- Public APIs with diverse consumers
- Web applications (universal browser support)
- Simple CRUD operations
- Teams unfamiliar with Protocol Buffers

33. How do you handle long-running operations?

Use the asynchronous request pattern:

1. Client sends request
2. Server returns 202 Accepted with a status URL
3. Client polls the status URL until completion

```
POST /reports

Response: 202 Accepted

Location: /reports/abc123/status


GET /reports/abc123/status

Response: {"status": "processing", "progress": 45}


GET /reports/abc123/status

Response: {"status": "completed", "result_url": "/reports/abc123"}
```


34. What is the Saga pattern?

Saga handles distributed transactions across microservices without locking. Each service executes a local transaction and publishes an event. If a step fails, compensating transactions undo previous steps.

```
Order Created → Payment Processed → Inventory Reserved → Order Confirmed

                                  ↓ (failure)

                    Release Payment ← Cancel Order
```

35. What are common JWT security vulnerabilities?

Algorithm confusion Attacker changes RS256 to HS256 and signs with the public key. Fix: always whitelist algorithms.

```
# Vulnerable

jwt.decode(token, key)


# Secure

jwt.decode(token, key, algorithms=["RS256"])
```


"none" algorithm: Attacker removes signature entirely. Fix: reject "none" algorithm.

Backend Developer REST API Questions

These questions focus on server-side implementation details, database optimization, and performance patterns that backend engineers encounter daily.

36. What is the N+1 query problem?

The N+1 problem occurs when you fetch N records and then execute a separate query for each record's relationship.

```
# N+1 Problem: 101 queries for 100 users

users = User.query.all()  # 1 query

for user in users:

    print(user.posts)     # 100 queries
```


Solutions:

Django: Use select_related() for foreign keys, prefetch_related() for many-to-many.

```
# 2 queries instead of 101

users = User.objects.prefetch_related('posts').all()
```

SQLAlchemy: Use joinedload() or selectinload().

37. How do you configure connection pooling?

Connection pooling reuses database connections instead of creating new ones for each request.

```
# SQLAlchemy
engine = create_engine(
    "postgresql://user:pass@host/db",
    pool_size=5,          # Permanent connections
    max_overflow=10,      # Temporary overflow
    pool_timeout=30,      # Wait timeout
    pool_pre_ping=True    # Validate before use
)
```


38. What caching strategies do you know?

There are three primary caching strategies, each optimized for different access patterns:

Strategy	Description	Use Case
Cache-Aside	**App checks cache, then DB**	**Read-heavy, tolerates staleness**
Write-Through	**Write to cache and DB together**	**Consistency critical**
Write-Behind	**Write to cache, async DB update**	**High write throughput**

39. How do you implement webhook signature verification?

Webhook signature verification ensures the request actually came from the expected sender. Here's an implementation using HMAC:

```
import hmac

import hashlib


def verify_webhook(payload: bytes, signature: str, secret: str) -> bool:

    expected = 'sha256=' + hmac.new(

        secret.encode(),

        payload,

        hashlib.sha256

    ).hexdigest()

    return hmac.compare_digest(expected, signature)
```


40. What metrics indicate API health?

You should monitor these four key metrics to assess your API's health:

Metric	Description	Alert Threshold
P50 Latency	**Median response time**	**Baseline**
P99 Latency	**Slowest 1%**	**> 2s for 5 min**
Error Rate	**% of 4xx/5xx**	**> 1% for 5 min**
Throughput	**Requests per second**	**Capacity planning**

Full Stack Developer REST API Questions

These questions test your understanding of both client and server concerns, particularly browser security, state management, and cross-origin communication.

41. When does a browser send a CORS preflight request?

Browsers send an OPTIONS preflight for:

- Methods other than GET, HEAD, POST
- Custom headers (Authorization, X-API-Key)
- Content-Type other than form-urlencoded, multipart/form-data, text/plain

42. How do you configure CORS securely?

Secure CORS configuration requires explicitly whitelisting origins and carefully handling credentials. Here's a secure Express.js setup:

```
// Express.js
app.use(cors({
  origin: ['https://yourdomain.com'],  // Never use * with credentials
  credentials: true,
  methods: ['GET', 'POST', 'PUT', 'DELETE'],
  allowedHeaders: ['Content-Type', 'Authorization']
}));
```

Powered By

Never use wildcard (*) with credentials. Never allow null origin.

43. Where should you store tokens on the client?

Token storage involves security trade-offs between XSS and CSRF vulnerabilities"

Storage	XSS Risk	CSRF Risk	Recommendation
localStorage	High	None	Avoid for tokens
HttpOnly Cookie	Low	Requires mitigation	Recommended
In-memory	Low	Low	Best for access tokens

Best practice: Store refresh tokens in HttpOnly cookies with SameSite=Strict. Keep access tokens in memory.

44. When should you use WebSockets versus SSE?

The choice depends on whether you need bidirectional communication and how you want to handle reconnection:

Feature	WebSockets	SSE
Direction	Bidirectional	Server to client
Reconnection	Manual	Automatic
Binary data	Yes	No

Use SSE for notifications, live feeds, stock tickers. Use WebSockets for chat, multiplayer games, collaborative editing.

45. How do you handle file uploads?

For large files, use chunked uploads with progress tracking:

```
async function chunkedUpload(file, chunkSize = 5 * 1024 * 1024) {
```

```
  const totalChunks = Math.ceil(file.size / chunkSize);

  for (let i = 0; i < totalChunks; i++) {
    const chunk = file.slice(i * chunkSize, (i + 1) * chunkSize);
    await uploadChunk(uploadId, i, chunk);
  }
}
```

Powered By

Scenario-Based REST API Questions

These questions evaluate your ability to apply REST principles to real-world problems. Interviewers want to see your design process and how you make architectural trade-offs.

46. Design a REST API for a hotel booking system

A hotel booking system needs to handle searches, availability checks, and reservations while preventing double-bookings. Here are the core resources:

Resources:

- /hotels - List and search hotels
- /hotels/{id}/rooms - Available rooms
- /bookings - Create and manage bookings
- /guests/{id} - Guest profiles

Key decisions:

- Use cursor-based pagination for hotel search
- Implement optimistic locking for room availability
- Return 409 Conflict for double-booking attempts
- Use idempotency keys for payment processing

47. How would you implement soft deletes?

Add a deleted_at timestamp instead of removing records:

```
@app.delete("/users/{user_id}")
def delete_user(user_id: int):
```

```
    user = db.get(user_id)
    user.deleted_at = datetime.utcnow()
    db.commit()
    return Response(status_code=204)


@app.get("/users")
def list_users(include_deleted: bool = False):
    query = User.query
    if not include_deleted:
        query = query.filter(User.deleted_at.is_(None))
    return query.all()
```

Powered By

48. How do you migrate from v1 to v2 without breaking clients?

Use the Strangler Fig pattern:

1. Deploy v2 alongside v1
2. Route new features to v2
3. Gradually migrate existing endpoints
4. Communicate deprecation via Sunset header
5. Remove v1 after migration period

49. How would you design a search API with complex filtering?

For a product search with multiple filters:

```
GET /products?q=laptop&category=electronics&price[gte]=500&price[lte]=2000&brand=apple,dell&in_stock=true&sort=-rating&limit=20&cursor=abc123
```

Powered By

Design decisions:

- Use Elasticsearch for full-text search (relational DBs struggle with fuzzy matching)
- Implement faceted search to show available filter options

- Return filter counts so users know how many results each filter produces
- Cache popular search combinations

50. How do you handle API versioning in a microservices architecture?

Options:

1. **Gateway-level versioning**: API gateway routes /v1/* to old services, /v2/* to new
2. **Service-level versioning:** Each service handles its own version negotiation
3. **Consumer-driven contracts:** Use Pact or similar tools to ensure compatibility

For most teams, gateway-level versioning provides the cleanest separation.

51. How would you implement rate limiting for different user tiers?

The solution involves defining tier-specific limits and checking them using Redis counters:

```
RATE_LIMITS = {
    "free": {"requests": 100, "window": 3600},
    "pro": {"requests": 1000, "window": 3600},
    "enterprise": {"requests": 10000, "window": 3600}
}


async def rate_limit_middleware(request: Request):
    user = get_current_user(request)
    tier = user.subscription_tier
    limits = RATE_LIMITS[tier]

    key = f"rate:{user.id}:{current_hour()}"
    count = await redis.incr(key)

    if count == 1:
        await redis.expire(key, limits["window"])
```

```
    if count > limits["requests"]:
        raise HTTPException(
            status_code=429,
            headers={
                "X-RateLimit-Limit": str(limits["requests"]),
                "X-RateLimit-Remaining": "0",
                "Retry-After": str(seconds_until_reset())
            }
        )
```

Powered By

Behavioral REST API Questions

These questions assess your communication skills and how you handle real-world API challenges. Focus on demonstrating your problem-solving process and how you work with stakeholders.

52. Explain REST to a non-technical person

"Think of a REST API like a waiter in a restaurant. You (the customer) cannot go into the kitchen directly. Instead, you tell the waiter what you want from the menu, and they bring it to you. The menu is like the API documentation, listing what you can order. The waiter follows specific rules: you ask for something (GET), you order something new (POST), or you send something back (DELETE). If the kitchen is out of something, the waiter tells you (404 Not Found) so you do not wait forever."

This analogy works surprisingly well in stakeholder meetings.

53. Describe a time you optimized an API

Use the STAR method:

- **Situation**: "Our search API had 3-second response times during peak traffic"
- **Task**: "I needed to reduce latency without major architecture changes"
- **Action**: "I added Redis caching for frequent queries, implemented cursor-based pagination, and added database indexes"
- **Result**: "Response time dropped to 200ms, and we handled 5x more traffic"

54. How do you communicate breaking changes to API consumers?

1. **Announce early**: Give at least 6-12 months notice for major changes
2. **Use Sunset header**: Sunset: Sat, 31 Dec 2026 23:59:59 GMT
3. **Provide migration guides:** Document exactly what changes and how to adapt
4. **Run both versions:** Keep old version running during transition
5. **Monitor usage:** Track which clients are still on the old version

55. What documentation do you create for APIs?

Essential documentation includes:

- **OpenAPI/Swagger spec:** Machine-readable contract
- **Getting started guide:** First API call in under 5 minutes
- **Authentication guide:** How to obtain and use credentials
- **Error reference:** All possible error codes and how to handle them
- **Changelog:** What changed in each version
- **Rate limit documentation:** Limits per tier and how to handle 429s

56. When would you NOT use REST?

REST is not always the right choice. Knowing when to use alternatives shows architectural maturity. I have seen teams force REST into use cases where it creates more problems than it solves.

Scenario	Better Alternative
Real-time chat or gaming	**WebSockets**
High-performance internal services	**gRPC**
Complex nested data queries	**GraphQL**
Event-driven architectures	**Message brokers (Kafka, RabbitMQ)**
Bidirectional streaming	**gRPC bidirectional or WebSockets**

REST API Quick Reference

Use this section as a quick lookup when you need to verify HTTP method properties, status codes, or common patterns during interview preparation or real-world development.

HTTP Methods Comparison

This table summarizes the key properties of each HTTP method, helping you choose the right one for your use case.

Method	Safe	Idempotent	Cacheable	Typical Use
GET	**Yes**	**Yes**	**Yes**	**Retrieve resource**
POST	**No**	**No**	**Conditional**	**Create resource**
PUT	**No**	**Yes**	**No**	**Replace resource**
PATCH	**No**	**No**	**Conditional**	**Partial update**
DELETE	**No**	**Yes**	**No**	**Remove resource**
HEAD	**Yes**	**Yes**	**Yes**	**Get headers only**
OPTIONS	**Yes**	**Yes**	**No**	**CORS preflight**

Status Codes Reference

Here are the most commonly used HTTP status codes, organized by category. Focus on understanding when to use each rather than memorizing the full specification.

2xx Success

- **200 OK**: Successful GET, PUT, PATCH
- **201 Created**: Successful POST (include Location header)
- **204 No Content**: Successful DELETE

4xx Client Errors

- **400 Bad Request**: Malformed syntax
- **401 Unauthorized**: Authentication required
- **403 Forbidden**: Authenticated but not authorized
- **404 Not Found**: Resource does not exist
- **409 Conflict**: State conflict
- **422 Unprocessable Entity**: Validation error
- **429 Too Many Requests**: Rate limited

5xx Server Errors

- **500 Internal Server Error**: Generic server failure
- **502 Bad Gateway**: Upstream server error
- **503 Service Unavailable**: Temporarily overloaded

Common Mistakes to Avoid

Even experienced developers fall into these traps. Watch out for these common pitfalls when designing or implementing REST APIs:

- **Using verbs in URLs** (e.g., /getUsers instead of /users)
- **Returning 200 OK for error states**, which breaks client-side error handling
- **Confusing 401 and 403** (Authentication vs. Authorization)
- **Over-nesting URLs** beyond two levels
- **Storing tokens in localStorage** (creates XSS vulnerabilities)
- **Ignoring idempotency** for POST requests
- **Returning generic error messages** without actionable details